# 🎬 Final Project: Cinephile Movie Database

Congratulations on completing the React Bootcamp! You've mastered the fundamentals of building data-driven applications. Now, it's time to take off the training wheels and build your first solo project from scratch.

### The Objective

Your task is to build **Cinephile**, a movie discovery dashboard. This project will require you to combine everything you've learned—from managing component state to fetching real-world data from an external API.

## 🚀 Key Features to Implement

1. **The Movie Grid:** Render a list of movies fetched from a source (Use OMDb API).
    - Each movie must show its Title, Year, Rating, and Poster.
    - If the movies don't fit in a single page, scrolling up will reveal more.
    - **Hint**: The "*search"* api provides more data than the *"get by id/title"* api, that you might need in order to complete the tasks.
2. **Live Search:** Create a search bar that filters the list of movies as the user types.
3. **A Details View:** When a user clicks a movie card, the app should switch from the "Home" view to a "Details" view. Show the plot summary, the cast, and a backdrop.
4. **Loading & Error States:** Show a visual loader while the data is being fetched.
    - Handle cases where no movies are found with a friendly "No results" message.

## 🛠️ Technical Requirements

To pass the "Screen Test," your code must demonstrate the following skills:

- **Functional Components & Props:** Break your UI into reusable pieces (e.g., `MovieCard`, `SearchBar`, `Button`).
- **State Management (`useState`):** Manage the search query, the list of movies, and the current "view" (Home vs. Details).
- **Side Effects (`useEffect`):** Fetch the movie data when the app mounts or when the search query changes.
- **Conditional Rendering:** Seamlessly switch between the Loading, Error, Home, and Details states.

## 🌟 Bonus Challenges

- **Global Settings:** Use the **Context API** to allow users to toggle between "Dark Mode" and "Light Mode."
- **Auto-Focus:** Use `useRef` to automatically focus the search bar when the page loads.
- **Routing:** Use **React Router** to give every movie its own unique URL (e.g., `/movie/123`).

- **Memory:** Add an “Add to Watchlist” feature, that persists when you load the site
- **Performance**: Use debounce for the search mechanism
- **Data manipulation:** Add sort by year, rating, title (A-Z)
- **Loading states:** Replace the boring “Loading…” text with a skeleton and shimmering effect

**Remember:** Focus on the logic and data flow first. Once the "search" and "details" logic works, put on your designer hat and make it look like a blockbuster!

**Good luck, and happy coding!**